

Calgary Crime Data Analysis and LSTM Prediction

Project report: exploratory analysis and a neural network on community crime counts

Notebook: crime_data_analysis_and_neural_networks_final.ipynb | Author: Yash Jain

1. Introduction

The City of Calgary publishes crime statistics for each community through its open data portal. This project has two goals. The first is to understand the data: which crime categories and which communities appear most, and how that changes by year and by month. The second is to build a neural network that predicts the crime count and see how well it does.

I explored the data with a series of charts and then trained an LSTM network in Keras. The exploratory part gives a reasonable overview, although I found a mistake in how I counted crimes (section 4). The model reaches a test mean squared error of 4.82, but the way I set up the sequences means that result should not be read as a real forecast (sections 5 to 7).

2. Data

The dataset is the Community Crime Statistics file from the City of Calgary Open Data portal (Community_Crime_Statistics_20240522.csv). It has 70,661 rows and 5 columns, with no missing values:

Column	Description
Community	Community code or name
Category	Type of crime, nine categories
Crime Count	Number of crimes for that community, category and month
Year	2018 to 2024
Month	1 to 12

Each row is one combination of community, category and month. The counts are small: mean 2.86, median 2, standard deviation 3.66 and maximum 111. The minimum is 1, which tells us that months with zero crimes are not in the file. The data is therefore sparse: a missing row means no crime was recorded. The file is sorted alphabetically by community, and not by date, which matters a lot for the model.

The nine categories are Theft FROM Vehicle, Theft OF Vehicle, Break & Enter - Commercial, Break & Enter - Dwelling, Break & Enter - Other Premises, Assault (Non-domestic), Violence "Other" (Non-domestic), Street Robbery and Commercial Robbery. The data stops in spring 2024, so 2024 is only a partial year.

3. Preprocessing

Cleaning. Nothing needed fixing: no nulls, and the data types were already correct (text for community and category, integers for the rest).

Encoding. For the model, I label-encoded Community and Category with scikit-learn, so each name became an integer. These numbers have no real order, but the LSTM treats them as ordinary numeric inputs.

Scaling. None. Year is around 2,020, month is from 1 to 12, and the target ranges up to 111. This explains why the first training loss was about 800.

4. Exploratory analysis

An important caveat. The charts for communities, categories, years and months all use `value_counts` or `countplot`, which count the number of rows. They do not sum the Crime Count column. A row in which 40 crimes happened counts the same as a row with 1. The percentages below are therefore the share of records, not the share of crimes. I would redo them with a sum of Crime Count.

Communities

Beltline has the largest share of records among the top ten communities (11.4%), followed by Forest Lawn (10.7%) and Downtown Commercial Core (10.2%). Marlborough, Albert Park/Radisson Heights, Bowness, Falconridge, Pineridge, Bridgeland/Riverside and Dover make up the rest, all between about 9.4% and 10.0%, so the top ten are close to each other. The right-hand pie shows the ten communities with the fewest records. These have only a handful of records each (06B is the largest at 22.7% of that small group, then 02B and 02K at 13.6% each), so I would not call them the "safest" communities. They are probably small or non-residential areas.

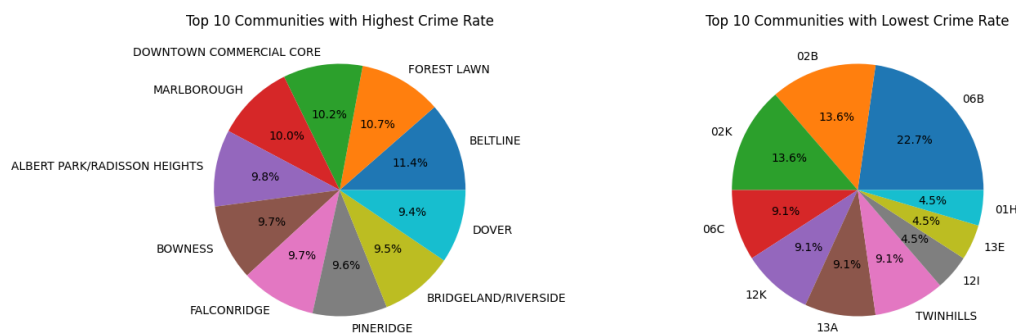


Figure 1. Communities with the most records (left) and the fewest records (right).

Crime categories

Theft FROM Vehicle is the most common category (21.7% of records), followed by Theft OF Vehicle (16.7%), Break & Enter - Commercial (13.8%), Assault (Non-domestic) (12.8%) and Violence "Other" (10.9%). Break & Enter - Dwelling (9.5%) and Break & Enter - Other Premises (8.5%) are lower, and robberies are rare: Street Robbery 3.7% and Commercial Robbery 2.5%.

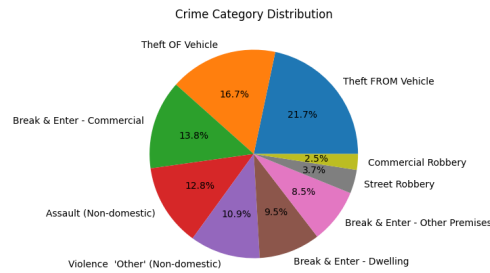


Figure 2. Share of records by crime category.

Years and months

From 2018 to 2023 the number of records per year is steady, between roughly 10,800 and 11,800, with 2018, 2019 and 2022 slightly higher. 2024 has about 3,100 because the data stops in spring. By month, January has the most records (about 6,400) and May and the last two months of the year the fewest (roughly 5,500 to 5,600). The differences are small.

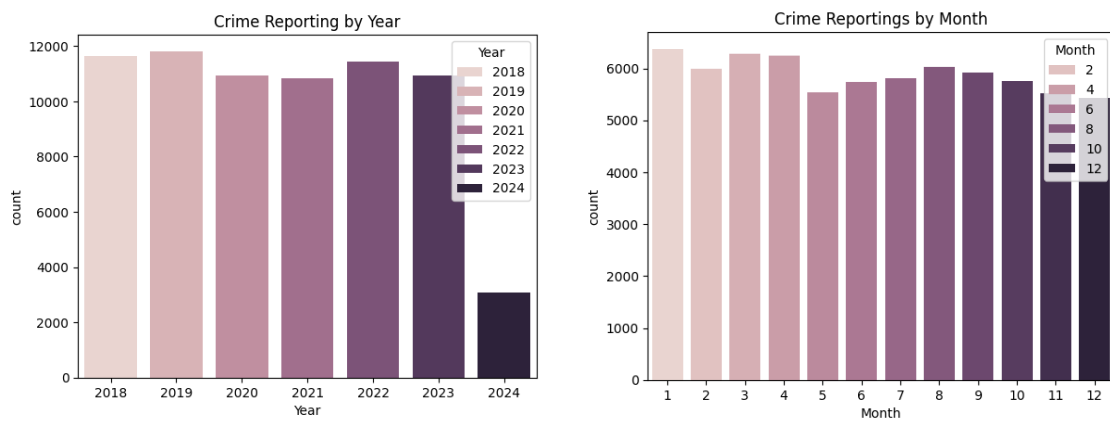


Figure 3. Number of records by year (left) and by month (right).

Years and categories

The mix of categories is consistent from year to year: Theft FROM Vehicle is first every year, and Commercial Robbery and Street Robbery are last. Theft OF Vehicle and Break & Enter - Commercial change their order a few times, but nothing dramatic stands out.

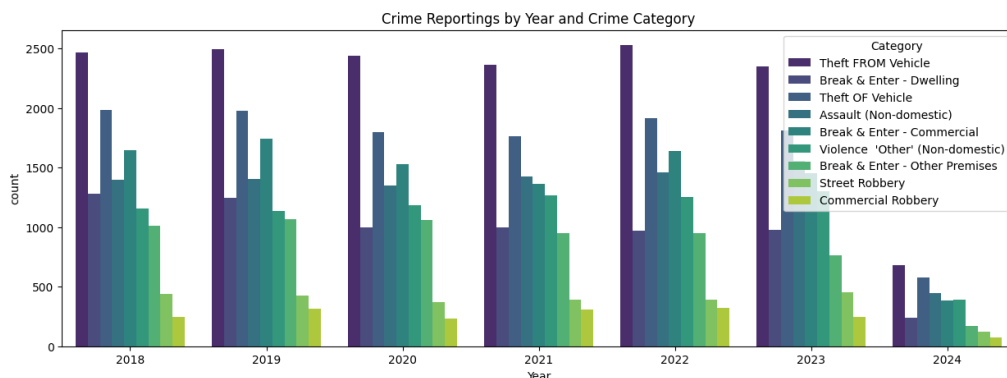


Figure 4. Number of records by year and crime category (2024 is a partial year).

5. Modelling

Sequences. I wrote a function that takes 3 consecutive rows of the dataframe (all 5 columns, including the current Crime Count) as one input, and uses the Crime Count of the following row as the target. This produces 70,658 sequences of shape (3, 5).

Split. The sequences were split at random: 70% training (49,460), 15% validation (10,599) and 15% test (10,599), with random_state 42.

Architecture. An LSTM layer with 50 units and relu activation, a dropout layer with a rate of 0.2, and one dense output neuron. The optimiser is Adam with a learning rate of 0.001, the loss is mean squared error, and I trained for 100 epochs with a batch size of 16. There was no early stopping, no tuning and no scaling.

6. Results

Metric	Value
Test MSE	4.82
Test RMSE	about 2.20
Training loss, last epoch	about 5.2 (with dropout active)
Validation loss, last epochs	about 4.6 to 4.9

The training loss starts at about 800 in the first epoch (because nothing is scaled) and falls to single digits by the second. After that it is nearly flat, and the validation loss is noisy but stable, which suggests the model stopped learning meaningfully long before epoch 100.

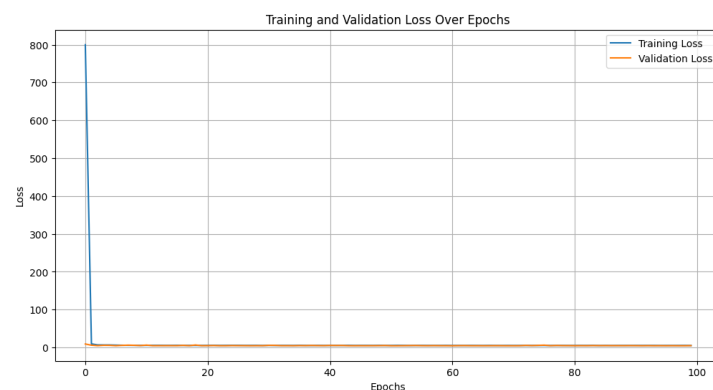


Figure 5. Training and validation loss over the 100 epochs.

An RMSE of about 2.2 crimes is relatively small compared with a standard deviation of 3.66 in the target. A model that always predicts the mean would have an MSE of about 13.4 (the variance), so the network is clearly better than a constant guess. I did not compare it with other, smarter baselines such as repeating the previous row's count.

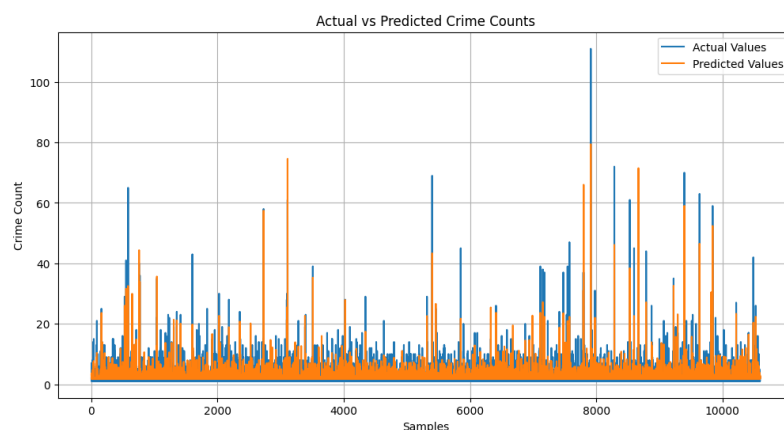


Figure 6. Actual and predicted crime counts on the test set.

The predictions follow the large spikes in the actual values quite well and are close for most small counts. The residuals are centred around zero, with a few large errors of 20 to 30 crimes on the biggest spikes.

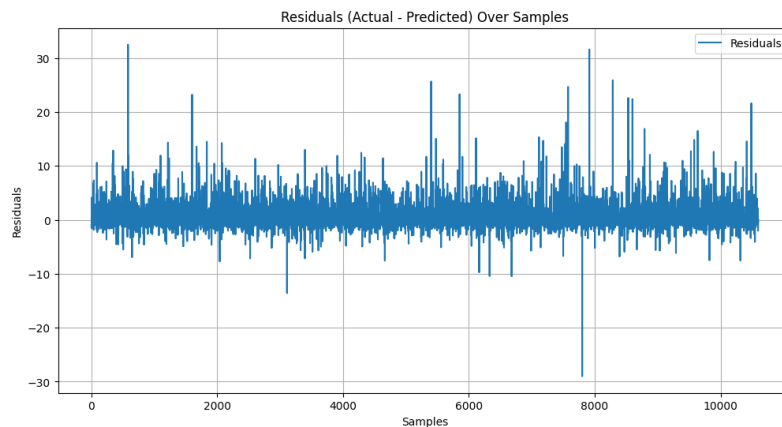


Figure 7. Residuals (actual minus predicted) on the test set.

7. Limitations

These matter a lot, and I think the second half of this project needs to be redone.

- **The sequences are not time series.** The file is sorted by community and category, not by date. Three consecutive rows are typically three different months or years of the same community and category. They are not three months in a row. The model is therefore predicting the count of a neighbouring row, not the next month. This is also a long way from the goal stated at the top of the notebook, which was to predict the number of crimes in the future.
- **Possible ordering effect.** From the first few rows, the data seems to be ordered by crime count within each community and category. If so, the count of the previous rows would say a lot about the count of the next row, which would explain why the model follows the spikes so well. I did not verify this, but it means the 4.82 should not be trusted as a sign of forecasting ability.
- **Leakage from the random split.** Windows of 3 rows overlap, so after a random split nearly identical windows end up in training, validation and test.
- **Rows are counted as crimes.** As explained in section 4, the charts count records rather than summing Crime Count.
- **Sparse data.** Months with zero crimes have no row, so they would need to be added back before building proper monthly series.
- **No optimisation.** The plan in the notebook includes optimising the model, but no hyperparameters were tuned, and there is no early stopping or scaling.
- **Text that does not match the charts.** My notebook names 13M as the safest community (22.7%), but the chart shows 06B. It also says 2022 had more records than 2018, while the bars show 2018 slightly higher.
- **Only one metric.** I reported MSE and no MAE, R^2 or baseline comparison. The actual-versus-predicted line plot is drawn over shuffled samples, so it does not show a trend over time.

- **Reported crime only.** The data counts crimes that were reported to the police, so it does not show how safe a community is. Communities also differ a lot in population and in how many people pass through them (for example, Beltline and the Downtown Commercial Core), and the data is not adjusted for that.

8. Conclusion

The exploratory analysis shows a consistent picture: vehicle-related theft and break-ins dominate the records, robberies are rare, and the yearly and monthly patterns are fairly stable. Beltline, Forest Lawn and the Downtown Commercial Core appear most often, though the top ten communities are close to each other and the counts are of records rather than crimes.

The LSTM reaches a test MSE of 4.82 (RMSE about 2.2), clearly better than predicting the mean. But because the sequences are made from rows that are not in time order, and the split is random, this is not a forecast of future crime. I would treat the neural network part as a first attempt that shows how not to set up the data, and the next version should start from a proper monthly time series.

9. Future work

- Sort by date and build a complete monthly series for each community and category, filling missing months with zero.
- Split by time (for example, train on 2018 to 2022, validate on 2023, test on 2024) and predict the following month, or several months ahead.
- Compare the LSTM with simple baselines (last month, same month last year, a moving average) and with gradient boosting.
- Scale the inputs and the target, use embeddings for community and category, and add early stopping.
- Redo the charts by summing Crime Count, and add crime per 1,000 residents using population data.
- Tune the number of units, the sequence length, the dropout rate and the learning rate.
- Use the model to produce actual future predictions, since the notebook never does.

10. Tools used

Python, pandas, NumPy, Matplotlib, Seaborn, scikit-learn, TensorFlow/Keras, Google Colab.